MODULE 3 · LECTURE 17

CS 6530 Applied Cryptography

Elliptic-Curve Digital Signatures

Module 3 – Public-Key Cryptography

From EC Key Agreement to EC Signatures

ELLIPTIC-CURVE FOUNDATION

$$Q = dG$$

KEY AGREEMENT

ECDH / X25519

Purpose: Establish shared key material

Two parties use EC scalar multiplication to derive common key material.

DIGITAL SIGNATURES

ECDSA / EdDSA

Purpose: Authenticate signed data

Sign with private key • Verify with public key

INSIGHT

One EC foundation — different cryptographic purposes.

? THINK LIKE AN ENGINEER

For an EC signature, what will the private key and public key look like?

EC scalar multiplication can support both key agreement and digital signatures.

ECDSA

Keys and Ingredients

LONG-TERM KEY PAIR

Private key

d

SCALAR

scalar / integer

Public key

$Q = dG = (x_Q, y_Q)$

POINT

point on the elliptic curve

scalar multiplication with G

G is the agreed base point.

FOR EACH SIGNATURE

Message m → Digest $e = H(m)$

Per-signature secret

secret scalar used for this signature

! ECDSA nonce k

Secret • must not be reused

Unlike an AEAD nonce, the ECDSA nonce must remain secret.

? THINK LIKE AN ENGINEER

ECDSA already has the long-term secret d. Why does each signature need another secret k?

INSIGHT

The EC private key is a scalar; the corresponding public key is a point on the curve.

ECDSA uses private scalar d, public point Q = dG, and a per-signature secret scalar k.

ECDSA

Creating a Signature

1 HASH THE MESSAGE

$$m \rightarrow e = H(m)$$

Message → message digest, represented as an integer.

2 GENERATE THE SECRET

$$k, \quad 1 \leq k < n$$

Generate a unique per-signature secret k (the ECDSA nonce): randomly, or using an approved deterministic method.

3 CREATE THE EPHEMERAL EC POINT

$$R = kG = (x_R, y_R) \rightarrow \text{take } x \rightarrow r = x_R \bmod n$$

4 COMPUTE THE SECOND SIGNATURE COMPONENT

$$s = k^{-1}(e + rd) \bmod n$$

e: message digest · d: long-term private key
k: per-signature ECDSA nonce

SIGNATURE

$\sigma = (r, s)$

Not d, k, or the point R.

? THINK LIKE AN ENGINEER

The verifier knows neither d nor k. How can (r, s) be verified?

INSIGHT

k creates the one-time point R = kG; the private key d enters the signature through s.

ECDSA combines the message hash, private key and per-signature secret to produce the signature (r, s).

ECDSA

Verifying a Signature

VERIFIER HAS m, (r, s), Q — public: G, n

1-2 VALIDATE & HASH

$1 \le r, s \le n - 1$

Reject otherwise.

Hash: $e = H(m)$

3-4 INVERT & SCALE

Invert: $w = s^{-1} \pmod{n}$

$u_1 = ew \pmod{n}$

$u_2 = rw \pmod{n}$

5 COMPUTE THE VERIFICATION POINT

$$X = u_1 G + u_2 Q \quad = \quad (x_X, y_X)$$

6 CHECK

$$x_X \bmod n \;=\; r$$

Equal → Signature VALID

Not equal → Signature INVALID

? THINK LIKE AN ENGINEER

Why should $u_1G + u_2Q$ reconstruct the point associated with the signer's nonce?

INSIGHT

The signer created (R = kG) using the secret nonce ((k); verification reconstructs the same point using only public information.

ECDSA verification combines the message and public key to reconstruct the signing relation and checks it against r.

Why Does Verification Work?

DERIVING THE VERIFICATION IDENTITY

Start: $s = k^{-1}(e + rd) \mod n$

$sk = e + rd \pmod n \quad \Rightarrow \quad k = s^{-1}(e + rd) \mod n$

With $w = s^{-1}$, $u_1 = ew$, $u_2 = rw$: $\quad k = u_1 + u_2 d \pmod n$

Multiply by G, use $Q = dG$: $\quad kG = u_1 G + u_2 dG = u_1 G + u_2 Q$

VERIFICATION IDENTITY

$$X = u_1 G + u_2 Q = (u_1 + u_2 d)G = kG = R$$

Therefore: $x_X \mod n = r$ (since $X = R$)

? THINK LIKE AN ENGINEER

If an ECDSA signature contains only (r, s), how large will the signature be?

256-bit curve order: r, s each ≈ 256 bits → 512 bits = 64 bytes (fixed-width r||s). DER encoding may add overhead.

INSIGHT

The verifier does not need (d): the public-key relation (Q = dG) turns the private-key contribution ($u_2 dG$) into the public term ($u_2 Q$).

A valid ECDSA signature makes the verifier reconstruct the same EC point R = kG used during signing.

TOY ECDSA

Sign a Message

Toy parameters for learning only — not cryptographically secure.

Curve: E: $y^2 = x^3 + 2x + 2 \pmod{17}$ $G = (5, 1)$, $n = \text{ord}(G) = 19$

GIVEN

Private key: $d = 7$

Public key: $Q = dG = 7G = (0, 6)$

Message hash: $e = H(m) = 13$

Per-signature secret: $k = 3$

1 COMPUTE r

$$R = kG = 3G = (10, 6)$$

$$r = x_R \bmod 19 = 10$$

2 COMPUTE s

$$3^{-1} \equiv 13 \pmod{19}$$

$$s = k^{-1}(e + rd) \bmod 19$$

$$= 13 \times 83 \bmod 19 = 15$$

SIGNATURE

$$\sigma = (r, s) = (10, 15)$$

? THINK LIKE AN ENGINEER

Can the verifier confirm (10, 15) using only Q = (0, 6) and the message hash?

INSIGHT

The point R = (10, 6) is used internally during signing; the signature contains only the scalars (r, s) = (10, 15).

The toy ECDSA signing process turns (e, d, k) into the two signature values (r, s).

TOY ECDSA

Verify the Signature

VERIFIER HAS (from the previous slide) $Q = (0, 6)$ $e = 13$ $(r, s) = (10, 15)$

1 COMPUTE w

$$w = s^{-1} = 15^{-1} \bmod 19$$

because $15 \times 14 = 210 \equiv 1 \pmod{19}$

$$\Rightarrow \quad w = 14$$

2 COMPUTE u1, u2

$$u_1 = ew = 13 \times 14 = 182 \equiv 11 \pmod{19}$$

$$u_2 = rw = 10 \times 14 = 140 \equiv 7 \pmod{19}$$

3 COMPUTE THE VERIFICATION POINT

$$X = u_1G + u_2Q = 11G + 7Q = 60G = 3G = (10, 6)$$

(using Q = 7G; G has order 19, so 60G = 3G)

4 FINAL CHECK

$$x_X \bmod 19 = 10 = r \quad \Rightarrow \quad \checkmark \text{ SIGNATURE VALID}$$

(X = (10, 6) = R — matches the point computed during signing)

INSIGHT: the verifier reconstructs R = (10, 6) without ever learning d = 7 or k = 3.

? **THINK LIKE AN ENGINEER**

What happens if the signer uses the same secret k = 3 for another message?

A valid ECDSA signature causes the verifier to reconstruct the same point R = kG used by the signer.

ECDSA NONCE FAILURE

Reuse the Nonce, Lose the Key

Continuing the toy example: $n = 19$, $d = 7$, $Q = (0, 6)$

TWO SIGNATURES, SAME k

Message 1 — already signed

$e_1 = 13$, $k = 3$ ⇒ $R = 3G = (10, 6)$

⇒ $(r, s_1) = (10, 15)$

Message 2 — nonce accidentally reused

$e_2 = 4$, $k = 3$ again ⇒ $R = 3G = (10, 6)$ again

⇒ $r = 10$ again, $(r, s_2) = (10, 12)$

Same r = 10 in both signatures!

same k → same R → same r

ATTACKER RECOVERS THE SECRETS

1 — Recover k

$k = (e_1 - e_2)(s_1 - s_2)^{-1} \mod n$

$= 9 \times 3^{-1} \bmod 19 = 9 \times 13 = 117 \equiv 3$

Nonce k = 3 RECOVERED

2 — Recover d

$d = (s_1 k - e_1) r^{-1} \mod n$

since $r^{-1} = 2$: $d = (15 \times 3 - 13) \times 2 = 64 \equiv 7$

PRIVATE KEY d = 7 RECOVERED

SAME NONCE → RECOVER k → RECOVER PRIVATE KEY d

? THINK LIKE AN ENGINEER

Does ECDSA fail only when k is exactly reused?

INSIGHT

Reusing (k) gives two signature equations with the same (rd) term; subtracting them eliminates the private-key term and exposes k

Reusing the ECDSA nonce can reveal both the nonce k and the private key d.

ECDSA NONCE SECURITY

What Must Be True About k?

1 — SECRET

k must remain secret

$s = k^{-1}(e + rd)$

If k known:

$d = (sk - e)r^{-1} \bmod n$

Known k → private-key recovery

2 — UNPREDICTABLE

k must be unpredictable to an attacker

Weak randomness, bias, or partial leakage can reveal information about (k) and enable attacks on d.

bias / leakage / predictability

↓

nonce information

↓

possible key recovery

3 — NOT REUSED

k must not be reused across different messages

Same k can be enough to recover the long-term private key.

k1 = k2

⇓

r1 = r2

⇓

recover k → recover d

k: (Nonce) SECRET | UNPREDICTABLE TO ATTACKER | NOT REUSED

? THINK LIKE AN ENGINEER

Can an EC signature derive its per-message secret without depending on fresh randomness for every signature?

INSIGHT

ECDSA security depends not only on the EC discrete-log problem, but also on protecting its per-signature secret.

Leakage, predictability, bias, or reuse of the ECDSA nonce can compromise the signing key.

NONCE GENERATION

From Random to Deterministic Nonces

1 — ECDSA WITH A FRESH RANDOM NONCE

CSPRNG → k → ECDSA signing

A fresh secret k can be generated for every signature.

weak / faulty randomness → weak k → possible key compromise

2 — ECDSA CAN ALSO USE A DETERMINISTIC NONCE

ECDSA itself can use deterministic nonce generation.

d + H(m) ⟶ [specified cryptographic derivation] ⟶ k

Deterministic ECDSA (e.g., RFC 6979) — determines k from the key & message; no fresh draw needed.

"No fresh signing randomness" ≠ "no randomness at all"

Secure randomness is still required when generating the long-term secret key.

ECDSA

fresh random nonce ⟶ deterministically derived nonce

EdDSA / Ed25519 — a different design

incorporates deterministic per-message nonce derivation into its construction

? THINK LIKE AN ENGINEER

How does EdDSA turn the private key material and message into a deterministic signature?

INSIGHT

Deterministic nonce generation is not unique to EdDSA: ECDSA can also derive its nonce deterministically.

Deterministic nonce derivation computes the per-message secret from the private key and message instead of drawing a fresh random nonce for every signature.

EdDSA / Ed25519

Creating a Signature

EdDSA — EC signature design using twisted Edwards curves.
Ed25519 — a specific EdDSA instance using Edwards25519 & SHA-512.

$G \leftrightarrow B$ (base pt) $\quad n \leftrightarrow L$ (order) $\quad d \leftrightarrow a$ (private scalar) $\quad Q = dG \leftrightarrow A = aB$ (public key)

$k \rightarrow R = kG \quad \leftrightarrow \quad r \rightarrow R = rB$ (per-message secret & point)

⚠ **Notation warning: ECDSA r ≠ EdDSA r**

ECDSA: r comes from the point R (r = x_R mod n).
EdDSA: r is the secret scalar (r = H(prefix||M) mod L), and R = rB.

Key material: seed → SHA-512 + processing → a (private scalar) + prefix (secret nonce-derivation material).
Public key: A = aB. (analogous to ECDSA Q = dG)

1 DERIVE THE SECRET

$r = H(\text{prefix}\|M) \mod L$

$R = rB$

Deterministic — no fresh nonce draw

2 BIND R, KEY, MESSAGE

$h = H(\text{enc}(R)\|\text{enc}(A)\|M) \mod L$

R: per-message point
A: public key
M: message

3 COMPUTE S & SIGN

$S = r + ha \mod L$

$\sigma = (R, S)$

32-byte R + 32-byte S = 64 bytes

? THINK LIKE AN ENGINEER

The verifier knows A, M, R, and S — but not the private scalar a. How can the signature be verified?

INSIGHT

Ed25519 makes deterministic nonce derivation part of its signature construction, avoiding dependence on fresh randomness for each signature.

An Ed25519 signature is a fixed 64 bytes, containing an encoded EC point R and a scalar S.

Verifying a Signature

VERIFIER HAS A, M, (R, S) — **public:** B, L

A: pub-key pt · M: msg · (R,S): sig | Recall: A=aB, S=r+ha mod L

1 RECOMPUTE THE CHALLENGE

$$h = H(\text{enc}(R) \,\|\, \text{enc}(A) \,\|\, M) \bmod L$$

2 CHECK THE EC RELATION

$$SB \;?=\; R + hA$$

Equal → Signature VALID

Not equal → Signature INVALID

WHY DOES THIS WORK?

$S = r + ha \Rightarrow SB = (r+ha)B = rB + haB$ [using $R=rB$, $A=aB$] $\Rightarrow$ $SB = R + hA$

RELATE TO ECDSA

ECDSA: $X = u_1G + u_2Q$, check $x_X \bmod n = r$ | Ed25519: $SB \;?=\; R + hA$

Both exploit the public-key relation created by EC scalar multiplication.

? THINK LIKE AN ENGINEER

Can we verify this relation with small numbers, without using the real Ed25519 curve?

INSIGHT

Ed25519 verification turns the scalar signing equation S = r + ha into a public EC-point equation.

Ed25519 verification checks whether the signature satisfies the public relation SB = R + hA.

TOY EdDSA

Sign and Verify

Toy arithmetic for understanding only — not Ed25519 parameters or hashing.

Curve: E: $y^2 = x^3 + 2x + 2 \pmod{17}$ $B = (5, 1)$, $L = \text{ord}(B) = 19$

KEYS $a = 7$ $A = aB = 7B = (0, 6)$

SIGN

1 — DERIVE r

Assume deterministic derivation gives r = 3

$\Rightarrow$ $R = rB = 3B = (10, 6)$

2 — OBTAIN CHALLENGE h

Assume the challenge hash gives h = 4

3 — COMPUTE S

$S = r + ha \mod L = 3+4\times7 = 31 \equiv 12 \bmod 19$

$\sigma = (R, S) = ((10,6), 12)$

Notation: ECDSA (r, s) vs. EdDSA (R, S)

VERIFY

Start: $SB \stackrel{?}{=} R + hA$

Left side: SB = 12B

Right side: R+hA = 3B+4×7B = 3B+28B = 31B

Since B has order 19: 31B = 12B

On the toy curve: 12B = (0, 11)

Therefore: $SB = R + hA = (0, 11)$

✓ SIGNATURE VALID

? THINK LIKE AN ENGINEER

If the signature mathematics is correct, what can still go wrong in a real EC implementation?

INSIGHT

The same EC scalar-multiplication foundation supports ECDSA and EdDSA, but their signature equations are different.

EdDSA verification works because S = r + ha becomes the public point relation SB = R + hA.

How Can EC Implementations Fail?

Secure EC mathematics ⇏ secure EC implementation

1 — INVALID-POINT ATTACK

ATTACKER → malicious point P′
(outside the expected group)
⇒ uses $d \cdot P'$ directly
⇓ ***observable result*** ⇓ ***info about d***

Response: point/input validation

2 — SMALL-SUBGROUP / COFACTOR

Large group vs. tiny cycle: $P_s \to 2P_s \to \cdots \to P_s$
Feed P_s into dP_s ⇒ few outputs
⇓ ***possible key leakage / failure***
Twist-related inputs: a related concern.

Response: follow cofactor/subgroup rules

3 — TIMING / SIDE-CHANNEL ATTACK

secret d, $P \longrightarrow$ [scalar mult.] $\longrightarrow dP$

TIME | POWER | MEMORY | FAULT

⇓ ***ATTACKER*** ⇓ ***info about the scalar***
Recall: Montgomery ladder, not a full fix

Response: constant-time, hardened code

4 — SIGNATURE NONCE FAILURE

k reused / predictable / leaked
⇓ nonce information
⇓ ***PRIVATE-KEY COMPROMISE***
Ed25519 avoids dependence on fresh per-signature randomness; other implementation risks remain.

Response: secure nonce derivation

? THINK LIKE AN ENGINEER

Across ECDH, X25519, ECDSA and Ed25519, what remains common—and what changes?

INSIGHT

The attacks target different boundaries: the input point, the group structure, the secret computation, or the signature nonce.

Secure EC implementations require protocol-appropriate input handling, subgroup-aware design, side-channel resistance and safe nonce handling.

ELLIPTIC-CURVE CRYPTOGRAPHY

What Should We Remember?

EC SCALAR MULTIPLICATION

KEY AGREEMENT
ECDH / X25519
Purpose: Establish shared key material

DIGITAL SIGNATURES
ECDSA / Ed25519
Purpose: Authenticate signed data

SAME FOUNDATION, DIFFERENT SIGNATURE DESIGNS

	ECDSA	Ed25519
Public key	Q = dG	A = aB
Nonce	random or deterministic	deterministic (built into design)
Signature	(r, s)	(R, S)
Size	≈64 B* (raw r‖s)	64 B (fixed)

**ECDSA encoded size can be larger; e.g., DER adds overhead.*

APPLICATIONS

Certificates | Code signing | Secure messaging | Blockchain systems | Signed models /artifacts

Choose the primitive + follow the protocol + use a hardened implementation

INSIGHT

One EC foundation supports multiple cryptographic purposes, but each construction has its own equations, assumptions and implementation requirements.

Elliptic-curve cryptography provides efficient key agreement and digital signatures, but security depends on the complete construction and its implementation—not only on the curve.

REFERENCES & ADDITIONAL READING

References

PRIMARY TEXTBOOKS

Jean-Philippe Aumasson — Serious Cryptography: A Practical Introduction to Modern Encryption

Chapter 12 — Elliptic Curves

Christof Paar & Jan Pelzl — Understanding Cryptography

Chapter 10 — Digital Signatures, §10.5 — The Elliptic Curve Digital Signature Algorithm (ECDSA)
(§10.4 DSA provides useful background to ECDSA.)

STANDARDS / TECHNICAL REFERENCES

RFC 8032 — Edwards-Curve Digital Signature Algorithm (EdDSA)
Includes Ed25519.

RFC 6979 — Deterministic Usage of DSA and ECDSA

RFC 7748 — Elliptic Curves for Security (X25519 / X448)

Additional Reading: "A Short History of Elliptic Curves and ECC" (see the separate reading note)